STOP "DOING RANDOM SHIT" IN LOWER KURAST: KEEPING YOUR SUPERSTITION AT BAY

SISTER EIR

Abstract. In unseeded single-player *Diablo II: Resurrected*, repeatedly opening the six "superchests" in Lower Kurast (LK) on `/players 7` is the best way to farm High Runes (Vex through Ber) for early progression and endgame runewords on a Holy Grail run. Due to the sheer length and right-skewed variance of the grind, players frequently adopt superstitious rituals (such as kicking logs or varying movement paths to "un-stick" the random number generator) or ineffective collateral-looting strategies to cope psychologically, which actively delays their desired payoff. Section 2 provides concrete, optimal farming guidance, giving exact expected run counts to cube up to every High Rune and major runeword (*Heart of the Oak*, *Call to Arms*, *Grief*, *Infinity*, and *Enigma*). Sections 3–7 detail all the math, showing why superstition, collateral farming, and `/players 5` harm players, and explaining how exact expected run counts are calculated using absorbing Markov chains.

1. Introduction

In single-player *Diablo II*, Lower Kurast is the undisputed best place for **early-progression** High Rune farming. Because popping chests requires no kill speed—needing only a 63% or 105% Faster Cast Rate *Teleport* Sorceress and zero endgame gear—a fresh solo self-found (SSF) or Holy Grail account can immediately farm Lower Kurast to bootstrap foundational runewords such as *Heart of the Oak* (Vex), *Call to Arms* (Ohm), *Grief* and *Fortitude* (Lo), *Infinity* (2 Ber), and *Enigma* (Jah + Ber). At the same time, Hell Lower Kurast superchests are governed by the `Act 3 (H) SuperCx` treasure class, which caps at `Runes 15` (**Ber, #30**) and **cannot drop Jah (#31), Cham (#32), or Zod (#33)** directly.

Once a single-player account has unlocked its core progression runewords in Lower Kurast, **endgame-progression** continues with hunting direct **Jah, Cham, and Zod** drops, and `tc87` set and unique items. While beyond our scope here, accepted strategies center around either **density farming** (killing monsters in high density areas), or **special monster** farming (such as ghost-type monsters or the Travincal Council members). In density farming, players favor clearing the Secret Cow Level (e.g., with a *Lightning Fury* Javazon), the Chaos Sanctuary, or the Worldstone Keep, particularly when "terrorized." In special monster farming, the Travincal Council drops runes at an unusually high rate, and with a *Find Item* ("Horker") Barbarian, the player can nearly double their drops, allowing quick runs on `/players 1` without being subject to no-drop on the horking step. Ghost-type monsters such as finger mages, ghosts, or gloams cannot drop equipment (weapons and armor), making them more likely to drop runes. They spawn with significant density in the Arcane Sanctuary as well as in the Chaos Sanctuary.

Few probabilistic grinding mechanics in action role-playing games have been scrutinized as closely as the "superchests" of Lower Kurast (LK) in Act III of *Diablo II: Lord of Destruction* (patch 1.13c+) and *Diablo II: Resurrected* (D2R). In single-player mode, where map layouts persist across sessions unless the character switches difficulty, an optimal Lower Kurast map spawns two campfires near the waypoint, each flanked by two huts containing a total of three special chests—yielding $m = 6$ superchests per run. Because the chest object seed generation

Date: October 31, 2026.

algorithm truncates its internal state consumption to a 16-bit space of $N_{\text{pat}} = 2^{16} - 2 = 65{,}534$ distinct patterns (excluding the two boundary seed values `0x0000` and `0xFFFF`), the exact combinatorial frequency of every High Rune (HR) pattern can be enumerated directly from the game's treasure class execution logic.

Nevertheless, a persistent gap separates the exact discrete mathematics of superchests from player intuition regarding stochastic processes. A striking example appears in recent community discourse. In the video *"Watch this BEFORE you run Lower Kurast"* [1], content creator `Dead_Sett` documents a brutal opening dry spell of 2,000 runs finding nothing higher than a Gul rune, with his first High Rune (a Vex) finally dropping on run 2,461. Misinterpreting a demonstration by `Ginger Gaming Mentor` [5] (*"D2R It's Raining Ber Runes"*)—which showed that artificially fixing the game's initial random number seed produces repeatable chest drops if the player executes identical frame-perfect actions—`Dead_Sett` conflates fixed-seed determinism with standard unseeded single-player play:

> *"As I prepared to go again, the key takeaway from GGM's video was that Lower Kurast on single player can have deterministic drops if things are done the same way sequentially. This is tied to seed! And being the clown that I am, I continued to find the same runes over and over again because my pathing was so precise. The high level? JUST DO RANDOM SHIT AND SEE WHAT HAPPENS."*

Alarmed that he had found three Gul runes during his drought and later lamenting (*"At run 2,842, I pick up yet another Lem rune. This is far too frequent for coincidence, I'm really concerned that something's wrong"*), `Dead_Sett` adopts the superstitious routine of kicking logs, flipping corpses, and varying his movement path. Over the remainder of his 4,554-run sample, he experiences a dramatic reversal of fortune—dropping a Lo on run 3,153, a *second* Lo just 23 runs later on run 3,176, a Sur on run 3,494, back-to-back Sur runes 23 runs apart on runs 4,390 and 4,413, and a Ber on run 4,554—cementing his conviction that "doing random shit" unlocked the PRNG.

2. What Actually Matters in Lower Kurast

For a single-player runner grinding toward *Heart of the Oak* (Vex), *Call to Arms* (Ohm), *Grief / Fortitude* (Lo), *Infinity* (2 Ber), or *Enigma* (Jah + Ber = 3 Ber equivalents when cubed in Lower Kurast), there is no secret ritual that improves superchest luck. Every actionable result in this paper boils down to four practical truths:

2.1. How Long It Will Take to Cube Up to Target Runes and Runewords.

Every rune from Pul (#21) through Ber (#30) upgrades in the Horadric Cube via a uniform 2-to-1 rule (2 Pul → 1 Um, …, 2 Sur → 1 Ber). Table 1 reports the **exact expected number of 6-chest Lower Kurast runs** (and standard deviation) required to acquire each High Rune and major endgame runeword when picking up and cubing lower runes, derived via our absorbing Markov chain in Section 6. At a brisk, sustainable single-player pace of $\tau_0 = 25$ seconds per run (≈ 144 runs/hour), farming on `/players 7` while cubing Pul+ requires an average of:

- **Vex (headliner for *Heart of the Oak*): 861.7** runs (≈ **6.0** hours; 692.9 runs on naive spreadsheets; +35.8 runs to 897.5 in Pul-unit value when including the Pul rune required by *HOTO*).
- **Ohm (headliner for *Call to Arms*): 1,057.8** runs (≈ **7.3** hours; 847.9 runs on naive spreadsheets; +183.1 runs to 1,240.9 in Pul-unit value when including the Mal + Ist required by *CTA*).
- **Sur (1/2 Ber): 1,037.4** runs (≈ **7.2** hours; 826.2 runs on naive spreadsheets).

- **Lo (headliner for *Grief* / *Fortitude*): 1,569.9** runs (≈ **10.9** hours; 1,294.0 runs on naive spreadsheets; +46.2 runs to 1,616.1 in Pul-unit value when including the Mal required by *Grief*).
- **Ber (1×): 1,558.0** runs (≈ **10.8** hours; 1,268.5 runs on naive spreadsheets).
- ***Infinity* (2 Ber): 2,865.3** runs (≈ **19.9** hours; 2,537.1 runs on naive spreadsheets; +28.8 runs to 2,894.1 in Pul-unit value when including Mal + Ist).
- ***Enigma* (3 Ber): 4,136.4** runs (≈ **28.7** hours; 3,805.6 runs on naive spreadsheets).

A note on full runeword recipes and downward cubing: Because runes cannot be cubed downward in *Diablo II*, a player who drops an early natural Ohm (or Vex + 2 Gul) before finding the supporting Mal + Ist for *Call to Arms* cannot break that Ohm downward; if farmed strictly from Pul–Ist drops in Lower Kurast without visiting the Hell Countess, waiting for the dedicated Mal + Ist slots increases the exact multi-slot expectation for *Call to Arms* to 1,521.2 runs (and 1,670.3 runs for *Grief*'s Lo + Mal). By contrast, over a 2,865-run *Infinity* grind, leftover Pul–Ist runes from boundary overshoot almost always cover the Mal + Ist slots automatically (+1%). Notice the rightmost column of Table 1: **standard community spreadsheets underestimate single-rune grinds by** 21% **to** 26% (275 to 290 missing runs on Lo and Ber!). As we prove in Section 6.3, naive spreadsheets divide 1.0 target rune by the average fractional rune value dropped per run—implicitly assuming you never have any "leftover change" sitting in your stash when your final rune drops. In reality, whenever you already hold $\frac{7}{8}$ of a Lo rune in cubed Ohms/Vexes/Guls and a natural Lo drops from a chest, those partial runes in your stash contributed nothing to finishing that first Lo (though they roll over as a head start toward your *next* runeword, which is why the overshoot penalty shrinks from +22.8% on a single Ber to +12.9% on *Infinity* and +8.7% on *Enigma*).

TABLE 1. Exact expected 6-chest Lower Kurast runs $\mathbb{E}[R]$ (and standard deviation $\mathrm{SD}(R)$) to acquire a target rune or endgame runeword via direct drops vs. Horadric Cubing, alongside the naive linear-drift approximation $W/\mathbb{E}[\Delta V_{\mathrm{run}}]$.

	`/players 5`			`/players 7`			
Target Goal r^*	Direct	Cube Gul+	Cube Pul+ (SD)	Direct	Cube Gul+	Cube Pul+ (SD)	Naive Drift (Overshoot)
Mal (#23)	1,092.7	—	662.5 (519.6)	769.6	—	**559.4** (464.8)	468.8 (+19.3%)
Ist (#24)	2,100.9	—	876.5 (604.7)	2,023.1	—	**768.2** (535.4)	640.6 (+19.9%)
Gul (#25)	2,184.9	2,184.9	1,076.4 (730.3)	993.4	993.4	**699.0** (542.7)	559.4 (+25.0%)
Vex (#26)	1,820.8	1,444.7	1,121.4 (818.5)	1,820.8	1,058.6	**861.7** (606.0)	692.9 (+24.4%)
Ohm (#27)	5,461.6	2,021.9	1,641.1 (1,016.6)	2,184.9	1,202.1	**1,057.8** (717.6)	847.9 (+24.8%)
Lo (#28)	2,731.0	1,885.9	1,714.2 (1,223.2)	5,461.6	1,753.0	**1,569.9** (928.9)	1,294.0 (+21.3%)
Sur (#29)	2,023.1	1,511.0	1,464.4 (1,164.1)	1,214.0	1,060.3	**1,037.4** (877.8)	826.2 (+25.6%)
Ber (#30)	2,731.0	1,603.3	1,576.9 (1,183.9)	5,461.6	1,591.9	**1,558.0** (1,035.0)	1,268.5 (+22.8%)
Infinity (2 Ber)	—	2,949.5	2,898.5 (1,534.5)	—	2,930.1	**2,865.3** (1,359.3)	2,537.1 (+12.9%)
Enigma (3 Ber)	—	4,239.0	4,164.4 (1,825.0)	—	4,231.0	**4,136.4** (1,632.7)	3,805.6 (+8.7%)

2.2. **Why Collateral Looting Delays the Payoff (And Why Your Loot Filter Is Your #1 High Rune Generator).** Because Lower Kurast requires thousands of runs, players are frequently seduced by the **collateral-loot fallacy**: *"Since I'm opening all these chests anyway, I should pick up and cube every Flawless Gem, identify every Small and Grand Charm, and hoard low runes and crafting Jewels along the way."*

The flaw in this reasoning is that inventory management is not free. If your base run takes $\tau_0 = 25$ s, picking up an average of 2.5 collateral items per run at an amortized cost of 4 s per item (walking back into huts to click clutter, identifying charms, Tetrising inventory space,

walking to the stash, and cubing sets of three Flawless Gems) adds $\Delta\tau_{\text{loot}} = 10$ s per run ($\tau_1 = 35$ s). That single habit **slashes your High Rune arrival rate by** 28.6%, adding more than 8 **hours of pure inventory chore time** to an *Enigma* grind—the equivalent of throwing away 1,200 Lower Kurast runs (≈ 2.63 expected High Runes).

Conversely, the single highest-leverage "High Rune generator" in *Diablo II: Resurrected* is a **strict loot filter** (hiding gold, potions, equipment, gems, and every rune below Lem or Pul). Beyond eliminating click and stash-management time, a strict loot filter removes the visual parsing tax and psychological whiplash of scanning twenty white and orange labels in a hut and mistaking an *Ith* for an *Ist*, a *Sol* for a *Sur*, or an *Io* for a *Lo*—and it prevents high runes from being hidden off-screen when a `/players 7` chest eruption exceeds *Diablo II*'s 32-item ground label display cap. When trash is hidden, 99% of chest pops require zero cognitive effort, allowing you to cruise on effortless autopilot until a genuine mid-to-high rune flashes on screen (Section 5.2).

2.3. **Why Kicking Logs and Flipping Corpses Delays the Payoff.** In standard (unseeded) single-player play, each new game session initializes its random number generator from unknown system clock entropy. Think of the 65,534 possible superchest drop patterns as a **freshly shuffled deck of** 65,534 **cards sitting face-down in the dark**. Kicking a log, flipping a corpse, or taking a different teleport route before opening a chest simply cuts K cards off the top of that face-down deck. Because you cannot see the cards, cutting an unknown shuffled deck leaves the probability that the next card dealt is a High Rune **100% unchanged** (Proposition 3.1).

Why, then, did `Dead_Sett` feel that "doing random shit" after a 2,000-run drought cured his bad luck?

(1) **Droughts of** 2,000+ **runs are normal career variance, not a "stuck seed":** On `/players 7`, the probability of finding at least one Vex–Ber High Rune in a 6-chest run is 1 in 455.5 runs. Across a 15-High-Rune farming career, 1 **in** 6 **players (**17.0%**)** will suffer a dry spell of 2,000+ consecutive barren runs, and 1 **in** 15 **(**6.5%**)** will suffer a 2,460+ run drought (Section 4.2).

(2) **Memoryless regression to the mean:** Because the PRNG has no memory, the expected number of *additional* runs required to drop a High Rune after 2,000 failures is still 455.5 runs—and `Dead_Sett` dropped his Vex 461 runs after switching routines (run 2,461), followed by two textbook Poisson clusters (Section 5.1). Kicking logs did not unlock his loot table; it merely slowed down each run by $\Delta\tau$ seconds while regression to the mean did its job.

2.4. **Why There Is Never a Case for `/players 5`.** A persistent myth claims that players hunting a **Lo rune** (for *Grief* or *Fortitude*) should farm on `/players 5` instead of `/players 7` because `/players 5` has 4 direct Lo patterns per chest compared to 2 on `/players 7`.

As shown in Table 1, that advice only holds if you refuse to use the Horadric Cube. In reality, more than 76% of your progress toward a Lo rune comes from cubing Ohm, Vex, and Gul drops—and `/players 7` has **2.5**× as many Ohm patterns (5 vs. 2) and **2.2**× as many Gul patterns (11 vs. 5) as `/players 5`. Once cubing is included, **`/players 7` beats `/players 5` for acquiring a Lo rune on every metric**: lower expected runs (1,569.9 vs. 1,714.2), lower median wait (1,461 vs. 1,519), and a vastly safer 90th-percentile worst-case drought (2,820 vs. 3,401 runs)—while simultaneously dropping 66.7% more Sur runes toward your future *Infinity* or *Enigma* (Section 7).

Crucially, this dominance is not limited to the Lo rune: **`/players 7` strictly dominates `/players 5` to find or cube up to every single rune from Mal (#23) through Ber (#30)**. As detailed in Table 1 and Section 7.1, cubing from Pul+ requires 559.4 vs. 662.5 runs

for Mal (15.6% faster), 768.2 vs. 876.5 runs for Ist (12.4% faster), and 699.0 vs. 1,076.4 runs for Gul (35.1% faster). While `/players 5` yields a microscopic edge on isolated Pul (666.4 vs. 683.1 runs) and Um (637.6 vs. 671.9 runs), no informed player targets Pul or Um in Lower Kurast: the Hell Countess drops a direct Pul or Um at ~ 1 in 78 runs (or 1 Pul-equivalent every ~ 14 runs via her dedicated mid-rune table, vs. ~ 670 runs in LK). Dropping to `/players 5` for a trivial 34-run (5%) edge on an Um needlessly forfeits the massive +42% Mal, +120% Gul, +150% Ohm, +67% Sur, and +12.1% overall High Rune bonuses of `/players 7`.

With these four practical takeaways established, the remainder of the paper presents the formal mathematical models and proofs.

3. PRNG Architecture and Shift Invariance

In *Diablo II*, pseudo-random number generation is governed by a 64-bit state split into two 32-bit unsigned integers, $(s_n, c_n) \in \{0, 1, \ldots, 2^{32} - 1\}^2$, representing the current value and carry of a multiply-with-carry (MWC) / linear congruential generator (LCG) recurrence:

$$z_{n+1} = a s_n + c_n, \qquad s_{n+1} = z_{n+1} \bmod 2^{32}, \qquad c_{n+1} = \left\lfloor \frac{z_{n+1}}{2^{32}} \right\rfloor, \tag{1}$$

where $a = \texttt{0x6AC690C5} = 1{,}791{,}398{,}085$. Crucially, the generator is *unweighted* and *stateless with respect to item outcomes*: the transition map $T\colon \Omega \to \Omega$ on the state space Ω depends exclusively on the current internal state $\omega_n = (s_n, c_n)$ and never on what runes or items have dropped in prior games, nor on player "behavioral variety."

When a superchest is opened in patch 1.13c+ and D2R, the chest's internal seed initialization truncates the object seed to 16 bits, restricting the reachable drop sequences for a given chest state (unlocked or locked) to a finite set of patterns $\mathcal{P}$ with cardinality $|\mathcal{P}| = N_{\text{pat}} = 65{,}534$. Let $\mathcal{H} \subset \mathcal{P}$ denote the subset of patterns containing a High Rune (Vex–Ber).

In standard (unseeded) play, each time the player clicks "Save and Exit" and creates a new game in Lower Kurast, the initial game state $\omega_0 \in \Omega$ is initialized from high-resolution system clock/entropy sources unknown to the player. Thus, prior to opening a chest, the state ω_0 is modeled as a uniform random variable on Ω.

Suppose a player performs a set of extraneous actions (e.g., kicking logs, flipping corpses, or casting spells) before opening a superchest. Each such action invokes the PRNG transition map T some number of times, advancing the generator by $K \geq 0$ total steps to $\omega_K = T^K(\omega_0)$ before the chest pattern map $\pi\colon \Omega \to \mathcal{P}$ is evaluated.

Proposition 3.1 (Shift Invariance Under Extraneous PRNG Calls)**.** *Let $(\Omega, \mathbb{P})$ be a finite probability space equipped with the uniform probability measure $\mathbb{P}(\{\omega\}) = |\Omega|^{-1}$, and let $T\colon \Omega \to \Omega$ be the bijection induced by the PRNG state transition (1) on its periodic cycle. For any deterministic or random integer-valued number of extraneous PRNG invocations $K \geq 0$ independent of ω_0, the induced distribution of the chest pattern $\pi(T^K(\omega_0))$ is identical to that of $\pi(\omega_0)$:*

$$\mathbb{P}\big(\pi(T^K(\omega_0)) \in \mathcal{H}\big) = \mathbb{P}(\pi(\omega_0) \in \mathcal{H}) = \frac{|\mathcal{H}|}{N_{pat}}. \tag{2}$$

Proof. Since T is a bijection on the finite cycle Ω, so is the k-fold composition T^k for every fixed integer $k \geq 0$. Because ω_0 is uniformly distributed on Ω, for any subset $A \subseteq \Omega$ and any fixed $k \geq 0$,

$$\mathbb{P}\Big(T^k(\omega_0) \in A\Big) = \mathbb{P}\Big(\omega_0 \in T^{-k}(A)\Big) = \frac{|T^{-k}(A)|}{|\Omega|} = \frac{|A|}{|\Omega|} = \mathbb{P}(\omega_0 \in A).$$

Setting $A = \pi^{-1}(\mathcal{H})$ and conditioning on $K = k$ via the law of total probability yields

$$\begin{aligned}\mathbb{P}\big(\pi(T^K(\omega_0)) \in \mathcal{H}\big) &= \sum_{k=0}^{\infty} \mathbb{P}\Big(T^k(\omega_0) \in \pi^{-1}(\mathcal{H})\Big)\, \mathbb{P}(K = k) \\ &= \mathbb{P}\big(\omega_0 \in \pi^{-1}(\mathcal{H})\big) \sum_{k=0}^{\infty} \mathbb{P}(K = k) = \frac{|\mathcal{H}|}{N_{\text{pat}}}. \qquad \square\end{aligned}$$

Remark 3.2. Proposition 3.1 formalizes the classic casino fallacy of switching slot machines or altering button-press timing on an unseeded PRNG: applying an offset K to a uniformly distributed unknown state simply permutes which initial states map to $\mathcal{H}$ without altering the measure of the preimage. Because the player has zero information about ω_0, K is statistically independent of ω_0, and "doing random things" cannot increase the probability of rolling a High Rune. Note two standard idealizations in this model: (i) treating ω_0 as uniform on the periodic cycle Ω idealizes unseeded session initialization from system entropy, and (ii) writing $\mathbb{P}(\pi(\omega_0) \in \mathcal{H}) = |\mathcal{H}|/N_{\text{pat}}$ assumes the 16-bit chest projection $\pi\colon \Omega \to \mathcal{P}$ has uniform fiber sizes across the $N_{\text{pat}} = 65{,}534$ patterns (which holds up to a negligible $O(N_{\text{pat}}/|\Omega|) \approx 10^{-14}$ truncation remainder, whereas the first equality $\mathbb{P}(\pi(T^K(\omega_0)) \in \mathcal{H}) = \mathbb{P}(\pi(\omega_0) \in \mathcal{H})$ holds for any projection π).

4. Exact Chest Mechanics, Pattern Tables, and Dry Spell Probabilities

4.1. **Grounding in Actual Game Behavior: Locked vs. Unlocked Superchests.** A subtle methodological flaw pervades many early Lower Kurast guides. Historically, pattern tables on community forums were assembled via crowdsourced player screenshots. Because a Lower Kurast superchest can spawn in either an **unlocked** or **locked** state, and because a locked chest alters the chest's internal PRNG branching and drop-generation sequence when opened with a key (or by an Assassin, who opens locked chests natively without keys as an innate class trait), early manual tables inadvertently conflated or double-counted locked and unlocked variations—most notoriously inflating `/players 7` counts to 11 Sur patterns and 3 Ber patterns (27 total Vex–Ber patterns).

Later, reverse-engineering of the actual *Diablo II* chest generation code by `Fruit` [4] (`bonqen`, creator of the *Diablo II Pattern Generator*, `d2pg`) and archival verification by `fearedbliss` [3] established the exact deterministic pattern counts for both chest states across all player settings:

- Each superchest spawns **unlocked** with probability $q_{\text{un}} = 0.80$ (80%) and **locked** with probability $q_{\text{lk}} = 0.20$ (20%).[1]
- Assuming the player carries keys (or plays an Assassin) so that locked superchests are never skipped, the effective pattern count for rune r on a given player setting is the expectation over the chest's lock state:

$$w_r = q_{\text{un}}\,|\mathcal{H}_r^{\text{un}}| + q_{\text{lk}}\,|\mathcal{H}_r^{\text{lk}}| = 0.80\,|\mathcal{H}_r^{\text{un}}| + 0.20\,|\mathcal{H}_r^{\text{lk}}|. \tag{3}$$

[1] In `levels.txt`, the chest lock probability is determined by the area's base monster level (`monlvl1` = 22 in Normal Lower Kurast) via $\lfloor \texttt{monlvl1}/2 \rfloor + 8 = 19\%$ to 20%, as originally documented in the classic data file reference guides by `Nefarius` [7]. Notably, even though Hell Lower Kurast is an `alvl 80` zone (`MonLvl3Ex` = 80), a classic vanilla *Diablo II* engine quirk causes the chest lock initialization routine to unconditionally read the Normal difficulty column (`monlvl1` = 22) regardless of difficulty—otherwise Hell LK chests would spawn locked $\lfloor 80/2 \rfloor + 8 = 48\%$ of the time! Standard community reference tables derived from `d2pg` (such as `d2runes.io` by `TheHappyDev` [10]) weight unlocked and locked pattern counts by $(q_{\text{un}}, q_{\text{lk}}) = (0.80, 0.20)$, which produces the exact single-decimal weighted counts $w_r = 0.80\,|\mathcal{H}_r^{\text{un}}| + 0.20\,|\mathcal{H}_r^{\text{lk}}|$ reported in Table 2. Using 0.81/0.19 shifts expected run counts by less than 0.08%.

Table 2 presents the complete reverse-engineered pattern counts from Pul (#21) through Ber (#30) for both `/players 5` and `/players 7`.[2]

TABLE 2. Exact Lower Kurast superchest rune pattern counts (out of $N_{\text{pat}} = 65{,}534$) separated by chest state (Unlocked 80%, Locked 20%), following `Fruit`'s `d2pg` [4] (verified by `fearedbliss` [3] and tabulated by `TheHappyDev` [10]).

	`/players 5` (and `/players 6`)				`/players 7` (and `/players 8`)			
Rune	Unl.	Lck.	Weighted w_r	6-Chest Run Odds	Unl.	Lck.	Weighted w_r	6-Chest Run Odds
Pul (#21)	16	18	16.4	1 in 666.4	16	16	16.0	1 in 683.1
Um (#22)	11	11	11.0	1 in 993.4	10	11	10.2	1 in 1,071.2
Mal (#23)	10	10	10.0	1 in 1,092.7	14	15	14.2	1 in 769.6
Ist (#24)	5	6	5.2	1 in 2,100.9	5	7	5.4	1 in 2,023.1
Gul (#25)	5	5	5.0	1 in 2,184.9	11	11	11.0	1 in 993.4
Vex (#26)	6	6	6.0	1 in 1,820.8	6	6	6.0	1 in 1,820.8
Ohm (#27)	2	2	2.0	1 in 5,461.6	5	5	5.0	1 in 2,184.9
Lo (#28)	4	4	4.0	1 in 2,731.0	2	2	2.0	1 in 5,461.6
Sur (#29)	5	7	5.4	1 in 2,023.1	9	9	9.0	1 in 1,214.0
Ber (#30)	4	4	4.0	1 in 2,731.0	2	2	2.0	1 in 5,461.6
Vex–Ber	21	23	**21.4**	**1 in 510.8**	24	24	**24.0**	**1 in 455.5**
Pul–Ber	68	73	**69.0**	**1 in 158.7**	80	84	**80.8**	**1 in 135.6**

4.2. **Waiting-Time Distribution and Dry Spell Survival Analysis.** On `/players 7`, every rune from Gul (#25) through Ber (#30) has identical pattern counts in unlocked and locked chests (meaning that carrying keys on `/players 7` is about not forfeiting 20% of your superchests rather than shifting the conditional Gul–Ber drop distribution), totaling $w_{\text{HR}} = 6 + 5 + 2 + 9 + 2 = 24$ Vex–Ber patterns per chest:

$$p_{\text{chest}} = \frac{24}{65{,}534} \approx 3.66222 \times 10^{-4} \quad (1 \text{ in } 2{,}730.58 \text{ chests}). \tag{4}$$

For a standard run opening $m = 6$ superchests (modeled as independent draws given the MWC multiplier transitions between chest openings), the probability of obtaining at least one High Rune in a single run is

$$p_{\text{run}} = 1 - (1 - p_{\text{chest}})^6 \approx 2.19532 \times 10^{-3} \quad (1 \text{ in } 455.51 \text{ runs}). \tag{5}$$

Let $T \in \{1, 2, \dots\}$ denote the number of runs until the first High Rune drops. Under Proposition 3.1, successive runs are independent Bernoulli trials with parameter p_{run}, so $T \sim \text{Geom}(p_{\text{run}})$ with probability mass function $\mathbb{P}(T = n) = (1 - p_{\text{run}})^{n-1} p_{\text{run}}$. The expectation, standard deviation, and median of T are:

$$\mu_T = \mathbb{E}[T] = \frac{1}{p_{\text{run}}} \approx 455.51 \text{ runs}, \tag{6}$$

$$\sigma_T = \text{SD}(T) = \frac{\sqrt{1 - p_{\text{run}}}}{p_{\text{run}}} \approx 455.01 \text{ runs}, \tag{7}$$

$$\text{Med}(T) = \left\lceil \frac{\ln(1/2)}{\ln(1 - p_{\text{run}})} \right\rceil = 316 \text{ runs}. \tag{8}$$

[2]While `/players 8` shares the exact same treasure class no-drop divisor and superchest pattern table as `/players 7` (since no-drop only updates on odd player settings $1, 3, 5, 7$), running `/players 8` is strictly suboptimal because monster HP and attack damage scale upward on `/players 8`—an unnecessary hazard when blindly teleporting into a Cursed, Might-enchanted pack of Hell Buzzards inside a campfire hut.

Because $\sigma_T \approx \mu_T$, the waiting time T exhibits heavy right skewness. The probability of experiencing a "dry spell" of at least n consecutive runs with zero High Runes is the survival function

$$S(n) = \mathbb{P}(T > n) = (1 - p_{\text{run}})^n = (1 - p_{\text{chest}})^{6n}. \tag{9}$$

Table 3 evaluates $S(n)$ across benchmark run counts, alongside both the standardized linear score $z_{\text{lin}} = (n - \mu_T)/\sigma_T$ and the equivalent standard normal tail deviate ($z_{2\text{-sided}}$ such that $\mathbb{P}(|Z| > z_{2\text{-sided}}) = S(n)$ for $Z \sim \mathcal{N}(0,1)$).[3]

TABLE 3. Survival probabilities $S(n) = \mathbb{P}(T > n)$ of a dry spell lasting n consecutive 6-chest Lower Kurast runs without a Vex–Ber drop on `/players 7`.

Runs n	Chests $6n$	Survival $S(n)$	Odds (1 in $S(n)^{-1}$)	Linear $z = \frac{n-\mu_T}{\sigma_T}$	Normal Equiv. (2-sided)
316	1,896	49.93%	1 in 2.0	−0.31	0.68σ
456	2,736	36.71%	1 in 2.7	0.00	0.90σ
800	4,800	17.24%	1 in 5.8	+0.76	1.36σ
1,000	6,000	11.11%	1 in 9.0	+1.20	1.59σ
1,200	7,200	7.16%	1 in 14.0	+1.64	1.80σ
1,500	9,000	3.70%	1 in 27.0	+2.30	2.09σ
2,000	12,000	1.23%	1 in 81.1	+3.39	2.50σ
2,460	14,760	0.45%	1 in 222.8	+4.41	2.84σ

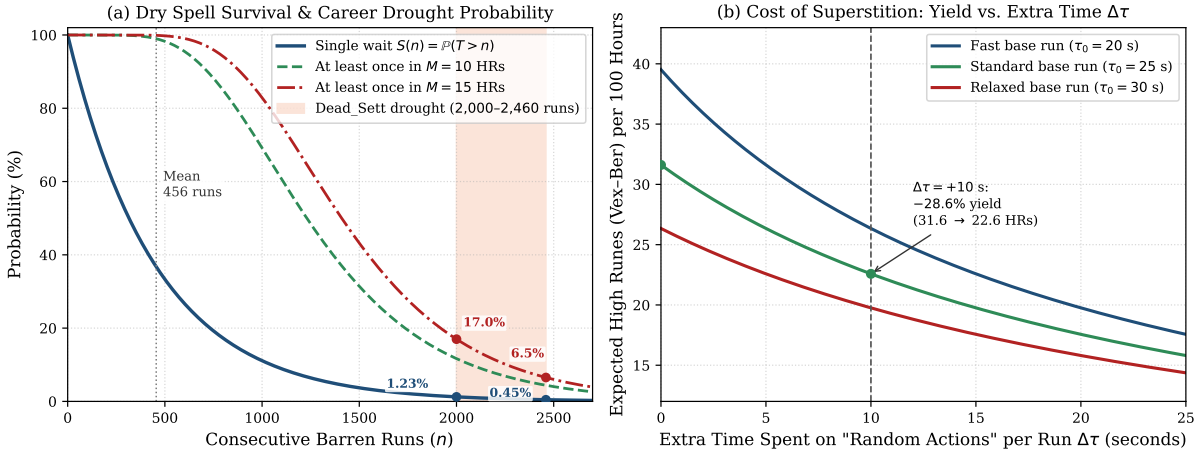


FIGURE 1. (a) Survival function $S(n) = \mathbb{P}(T > n)$ for a single High Rune wait on `/players 7` alongside the career probability $1 - (1 - S(n))^M$ of experiencing at least one drought of length $\geq n$ across $M \in \{10, 15\}$ High Rune drops. The shaded band highlights `Dead_Sett`'s 2,000–2,460 run dry spell. (b) Expected High Runes (Vex–Ber) per 100 hours of Lower Kurast farming as a function of extra time $\Delta\tau$ spent performing superstitious actions per run.

Table 3 and Figure 1(a) place `Dead_Sett`'s dry spell in exact statistical perspective:

[3]Following standard convention when translating p-values into "σ-levels," Table 3 reports the two-sided normal tail equivalent $z_{2\text{-sided}} = \Phi^{-1}(1 - S(n)/2)$. Because a dry spell $T > n$ is a one-sided upper-tail event, one may alternatively report the one-sided quantile $z_{1\text{-sided}} = \Phi^{-1}(1 - S(n))$, which equals 0.00σ at the median ($n = 316$), 2.25σ at $n = 2{,}000$, and 2.61σ at $n = 2{,}460$.

- Going 2,000 consecutive runs with zero Vex–Ber drops has single-wait probability $S(2{,}000) = 1.23\%$ (1 in 81.1), corresponding to a 2.50σ two-sided Gaussian tail event ($z_{\text{lin}} = +3.39$).
- Going 2,460 consecutive runs before dropping a Vex on run 2,461 has single-wait probability $S(2{,}460) = 0.45\%$ (1 in 222.8), corresponding to a 2.84σ two-sided Gaussian tail event ($z_{\text{lin}} = +4.41$).

Furthermore, a dedicated player completing a long-term goal (such as farming an Enigma or Infinity runeword) observes a sequence of independent waiting times $T_1, T_2, \ldots, T_M$ across M High Rune drops. The probability that *at least one* inter-drop gap exceeds n runs across M drops is

$$\mathbb{P}\left(\max_{1\le i\le M} T_i > n\right) = 1 - \big(1 - S(n)\big)^M. \tag{10}$$

For a player who farms $M = 15$ High Runes over a single-player career:

- The probability of encountering at least one 1,000-run dry spell is $1 - (1 - 0.11105)^{15} = \mathbf{82.89}\%$, and at least one 1,500-run dry spell is $\mathbf{43.20}\%$.
- The probability of encountering at least one 2,000-run dry spell is $1 - (1 - 0.01233)^{15} = \mathbf{16.98}\%$ (≈ 1 in 6 players), and at least one 2,460-run dry spell is $1 - (1 - 0.00449)^{15} = \mathbf{6.52}\%$ (≈ 1 in 15 players).

Across thousands of single-player Lower Kurast runners in the *Diablo II* community, a 1-in-223 single-wait drought (or 1-in-15 career drought) is guaranteed to happen to hundreds of players—and those who suffer it are precisely the ones most motivated to search YouTube for an explanation.

5. Regression to the Mean and the Cost of Superstition

We now formalize what transpires when a player, having endured a dry spell of $n_0 = 2{,}000$ runs, adopts the advice to "do random things" on run $n_0 + 1$ onward.

5.1. Memorylessness, Regression to the Mean, and Poisson Clustering: `Dead_Sett`'s Full 4,554-Run Ledger.

Let $T_{\text{rem}} = T - n_0$ denote the number of *additional* runs required to obtain a High Rune, conditioned on having failed for n_0 runs. By the memoryless property of the geometric distribution [2], for every $k \ge 1$,

$$\mathbb{P}(T - n_0 > k \mid T > n_0) = \frac{\mathbb{P}(T > n_0 + k)}{\mathbb{P}(T > n_0)} = \frac{(1 - p_{\text{run}})^{n_0 + k}}{(1 - p_{\text{run}})^{n_0}} = (1 - p_{\text{run}})^k = \mathbb{P}(T > k). \tag{11}$$

Consequently, conditioned on $T > n_0$, the remaining wait time T_{rem} is distributed identically to T:

$$\mathbb{E}[T - n_0 \mid T > n_0] = \mu_T \approx 455.51 \text{ runs}, \qquad \text{Med}(T - n_0 \mid T > n_0) = 316 \text{ runs}. \tag{12}$$

Examining `Dead_Sett`'s complete 4,554-run dataset alongside the exact `/players 7` pattern frequencies reveals three textbook statistical phenomena operating simultaneously:

1. The "Repeated Gul and Lem" Illusion. Before switching routines, `Dead_Sett` cited finding three Gul runes and recurring Lem runes by run 2,842 as proof that "precise pathing" was locking his character into repeated deterministic drops (*"far too frequent for coincidence, I'm really concerned that something's wrong"*). Aside from the category error of confusing artificially fixed seeds with unseeded play (Proposition 3.1), what is the actual expected number of Gul and Lem drops in $n = 2{,}842$ unseeded `/players 7` runs?

- **Gul (#25):** With $w_{\text{Gul}} = 11.0$ patterns per chest ($p_{\text{Gul,run}} \approx 1/993.4$), the expected number of Gul runes in 2,842 runs is

$$\mathbb{E}[N_{\text{Gul}}] = \frac{2{,}842}{993.4} = \mathbf{2.86} \text{ Gul runes},$$

and the Poisson probability of observing *exactly* 3 Gul runes is $e^{-2.86}(2.86)^3/3! =$ **22.3**%—making 3 Gul runes the literal *mode* (most probable single outcome) of the distribution!

- **Lem (#20):** On `/players 7`, Lem has 42 unlocked and 44 locked patterns ($w_{\text{Lem}} = 42.4$, or 1 in 258.0 runs), yielding an expectation of $\mathbb{E}[N_{\text{Lem}}] = 2{,}842/258.0 =$ **11.0** Lem runes by run 2,842. Finding "yet another Lem rune" on run 2,842 is not a glitch in the matrix; it is the law of large numbers doing its job.

2. Memoryless Post-Intervention Waiting Time. Around run $n_0 = 2{,}000$, after observing 0 Vex–Ber runes over his opening 2,000 runs, `Dead_Sett` switches to his "random behavior" protocol (kicking logs, flipping corpses, varying routes). By Proposition 3.1, this change has zero causal effect on p_{run}. By memorylessness, the conditional expected number of additional runs from run 2,000 to his first High Rune is $\mathbb{E}[T - 2{,}000 \mid T > 2{,}000] =$ **455.51** runs—and his Vex dropped on run 2,461, i.e., after **461** additional runs (within 1.2% of the exact unconditional expectation!).

3. Post-Drought Poisson Clustering and Full-Sample Reversion. Even more striking is what transpired across the second half of his dataset (runs 2,461 through 4,554), where he recorded seven High Runes in 2,094 runs (Table 4). Notice the extreme positive tail events embedded in

TABLE 4. Complete High Rune (Vex–Ber) drop ledger recorded by `Dead_Sett` across 4,554 Lower Kurast runs on `/players 7`.

Drop #	Run Number	High Rune	Inter-Drop Gap (Runs)	Ber-Equivalent Value
1	2,461	Vex (#26)	2,461 (461 post-switch)	0.0625 Ber
2	3,153	Lo (#28)	692	0.2500 Ber
3	3,176	Lo (#28)	**23**	0.2500 Ber
4	3,494	Sur (#29)	318	0.5000 Ber
5	4,390	Sur (#29)	896	0.5000 Ber
6	4,413	Sur (#29)	**23**	0.5000 Ber
7	4,554	Ber (#30)	141	1.0000 Ber
Total	**Runs** 1–4,554	**7 HRs**	**Mean gap** = 650.6	**3.0625 Ber**

this post-intervention streak:

- **Back-to-back natural Lo runes in** 23 **runs (runs** 3,153 **and** 3,176**):** On `/players 7`, a direct Lo rune has only 2 patterns per chest ($p_{\text{Lo,run}} \approx 1/5{,}461.6$). Conditioned on dropping a Lo on run 3,153, the probability of dropping a *second* natural Lo within the next 23 runs is

$$1 - \left(1 - \frac{1}{5{,}461.6}\right)^{23} \approx \mathbf{0.42}\% \quad (1 \text{ in } 238)\,.$$

Irony of ironies: `Dead_Sett`'s 23-run double-Lo burst is **more improbable in isolation on the lucky tail (**1 **in** 238**) than his entire** 2,460**-run opening drought was on the unlucky tail (**1 **in** 223**)!**[4]

[4]From a strict hypothesis-testing standpoint, there is a multiple-testing (*look-elsewhere*) asymmetry between these two events: whereas the opening drought $T_1 > 2{,}460$ occurred on the fixed first waiting time, the 23-run Lo-to-Lo and Sur-to-Sur gaps are post-hoc order statistics (minima) selected across 6 inter-drop intervals in a 4,554-run log. Across 6 independent inter-High-Rune gaps ($p_{\text{run}} \approx 1/455.51$), the probability that at least one gap is ≤ 23 runs is $1 - ((1 - p_{\text{run}})^{23})^6 \approx 26.2\%$ (and the chance of observing at least one same-tier pair within 23 runs across 7 drops is roughly 6%–9%). Far from weakening our argument, this look-elsewhere calculation is the essence of **Poisson clumping**: in a multi-thousand-run log, short back-to-back clusters are mathematically

- **Back-to-back Sur runes in 23 runs (runs 4,390 and 4,413), followed by a Ber 141 runs later (run 4,554):** Conditioned on the Sur at run 4,390, a second Sur within 23 runs ($p_{\text{Sur,run}} \approx 1/1{,}214.0$) has probability 1.88% (1 in 53), and capping the next 141 runs with a direct Ber ($p_{\text{Ber,run}} \approx 1/5{,}461.6$, probability 2.55%) delivers 2.0 Ber-equivalents in just 164 runs!
- **Full-sample balance:** Over the 2,554 runs following run 2,000, he collected 3.0625 Ber-equivalents from Vex–Ber alone, exceeding the 2,554-run Vex–Ber expectation (1.87 Ber-equivalents, or 2.01 Ber-equivalents including Pul–Gul) by +63.7%. Across all 4,554 runs, his 7 Vex–Ber High Runes (3.0625 Ber-equivalents, or ≥ 3.16 Ber-equivalents including his 3+ Gul runes) sit less than one standard deviation below the unconditional 4,554-run mean ($\mathbb{E}[N_{\text{HR}}] = 10.00 \pm 3.16$ High Runes; 3.34 expected Vex–Ber Ber-equivalents, or 3.59 expected Pul–Ber Ber-equivalents).

In statistical theory, this complete 4,554-run arc is the definitive illustration of **regression to the mean** and **Poisson clumping**: an extreme 2.5σ–2.8σ opening drought ($T_1 > 2{,}000$) is followed by an unconditional mean wait (461 runs to Vex) and two memoryless short-gap bursts (23 runs Lo-to-Lo, 23 runs Sur-to-Sur). In behavioral psychology, it replicates B. F. Skinner's (1948) classic pigeon experiments on *superstitious conditioning*: because the human brain experiences a 2,460-run drought as an eternity and two 23-run High Rune bursts after changing behavior as immediate causal feedback, the runner becomes unshakably convinced that kicking logs unlocked the loot table.

5.2. **The Strict Efficiency Penalty of "Doing Random Things" and the Collateral-Loot Fallacy.** While adding extraneous actions does not alter the per-run superchest probability p_{run}, it strictly increases the duration of each run. Let $\tau_0 > 0$ denote the mean duration (in seconds) of an optimized Lower Kurast run—typically $\tau_0 \in [20, 30]$ seconds in D2R including NVMe load times, achieved by (i) rolling a circular two-campfire map adjacent to the waypoint, (ii) popping chests and re-entering the waypoint through hut walls via the Sorceress's *Telekinesis* skill, and (iii) returning to Act IV (The Pandemonium Fortress) before "Save and Exit" (or stacking Faster Run/Walk and a *Harmony* bow's *Vigor* aura in Act III) to eliminate the long Kurast Docks town walk—and let $\Delta\tau > 0$ denote the mean time spent per run performing non-essential actions. Over a farming horizon of $H = 3{,}600$ s (1 hour), the expected arrival rate of High Runes is $\lambda(\tau) = (3{,}600/\tau)\, p_{\text{run}}$, so

$$\frac{\lambda(\tau_0 + \Delta\tau)}{\lambda(\tau_0)} = \frac{\tau_0}{\tau_0 + \Delta\tau} < 1 \quad \forall\, \Delta\tau > 0. \tag{13}$$

As illustrated in Figure 1(b), if an optimized runner averages $\tau_0 = 25$ s per run (≈ 144 runs/hour, $\mathbb{E}[\text{wait}] = 3.16$ hours per HR) and adds just $\Delta\tau = 10$ s of "random actions" per run ($\tau_1 = 35$ s), their High Rune acquisition rate drops by 28.6%, increasing the expected time per High Rune from 3.16 hours to 4.43 hours.

The Collateral-Loot Fallacy (Gems, Charms, and Jewels). A closely related cognitive bias in the Lower Kurast community—often invoked to rationalize slower, less disciplined runs—is the **collateral-loot fallacy**: *"Because you have to run Lower Kurast thousands of times anyway, you should pick up and process all flawless gems, small and grand charms, and crafting jewels along the way."*

While it is true that Lower Kurast superchests drop a massive volume of magic charms, jewels, and flawless gems, this argument treats the processing cost of collateral items as zero. In reality,

expected to occur somewhere, yet a human runner experiencing them after changing routines perceives a 1-in-238 miracle.

every item category added to the pickup filter introduces a multi-stage time overhead $\Delta\tau_{\text{loot}} > 0$ per run:

(1) **Visual parsing and acquisition ($\Delta\tau_{\text{click}}$):** Scanning densely cluttered ground text around six chests and walking back into huts to click Flawless Gems, Small Charms, Grand Charms, and Jewels.
(2) **Identification and cognitive evaluation ($\Delta\tau_{\text{eval}}$):** Opening the inventory or visiting Cain, casting *Identify*, and reading affix rolls—deciding whether a Small Charm is a worthless *Burly Small Charm* (+15 Defense) or a keeper like a *Shimmering Small Charm of Fortune* (+5% All Resistances, +7% Magic Find), or whether a Grand Charm rolled +7% Faster Run/Walk versus a +1 Paladin Combat Skills (*Lion Branded*) skiller.
(3) **Stash management and Horadric Cube overhead ($\Delta\tau_{\text{stash}}$):** Walking to the stash every few runs, sorting sets of three Flawless Gems to cube into Perfect Gems, transferring crafting Jewels to mules, and repeatedly rerolling `ilvl 80` Grand Charms.

Let n_{pick} denote the average number of collateral items picked up per run and c_{proc} the amortized time (in seconds) required to click, identify, evaluate, cube, and stash each item. Then $\Delta\tau_{\text{loot}} = n_{\text{pick}}\, c_{\text{proc}}$. Even a modest pickup rate of $n_{\text{pick}} = 2.5$ items/run at $c_{\text{proc}} = 4$ s per item (including periodic town/stash trips) injects $\Delta\tau_{\text{loot}} = 10$ s per run—imposing the **exact same** 28.6% **High Rune arrival-rate penalty** in (13) as kicking logs and flipping corpses! Over a 3,000-run Enigma project at $\tau_0 = 25$ s, that $\Delta\tau_{\text{loot}} = 10$ s overhead adds 8.33 hours of pure inventory management—equivalent to 1,200 lost Lower Kurast runs (≈ 2.63 expected Vex–Ber High Runes sacrificed).

Remark 5.1 (Skewness Fatigue, Consolation Prizes, and Informed Tradeoffs)**.** None of the foregoing analysis is intended to prescribe how players *ought* to enjoy a video game, nor to suggest that human runners are irrational for deviating from pure superchest isolation. Enduring a heavily right-skewed geometric waiting-time process ($\sigma_T \approx \mu_T \approx 455.5$ runs), in which hundreds of consecutive runs yield zero progress toward a High Rune, inflicts a genuine psychological toll. Leaning on "consolation prizes"—identifying charms, cubing flawless gems, or clicking a convenient log—injects frequent low-variance reinforcement into an otherwise barren reinforcement schedule. Moreover, because Lower Kurast superchests roll forced-magic cycles that produce an extraordinary concentration of Small Charms, Grand Charms, and Jewels, an early-progression solo self-found (SSF) character with empty inventory slots may rationally adopt a **two-phase filter strategy**: spending an initial phase identifying charms until baseline resistances and class "skillers" are secured, and then tightening the loot filter exclusively to runes for the remaining thousands of runs. Even players who laugh when content creators like `Sweet Phil` find a Vex rune under a log and proclaim *"always hit the poppables, I keep telling ya!"* inevitably make pragmatic or psychological concessions of their own (such as picking up Ort runes and chipped gems to recharge a *Teleport* staff via the Horadric Cube for rushing new non-Sorceress characters before they meet *Enigma*'s level 65 requirement). The mathematical point is not that collateral actions are forbidden, but that players should enter those tradeoffs *with their eyes open*: Poppables, gems, and charms do not "warm up" the PRNG, and every second $\Delta\tau$ spent away from the six superchests carries an exact, quantifiable opportunity cost $\tau_0/(\tau_0 + \Delta\tau)$ in High Runes per hour.

6. Cubing to a Given Target Rune

In practice, single-player farmers do not merely wait for a target rune r^* to drop directly; they also collect lower-tier runes and upgrade them in the Horadric Cube. A remarkable structural property of the *Diablo II* rune hierarchy is that whereas low-to-mid runes (El through Lem)

require *three* runes per upgrade ($3 \to 1$), every rune from **Pul (#21) through Ber (#30)** upgrades via a uniform **2-to-1 binary rule**:[5]

$$2 \times \text{Rune}(i) + \text{Flawed/Standard/Flawless Gem} \longrightarrow 1 \times \text{Rune}(i+1) \qquad (21 \le i \le 29). \quad (14)$$

6.1. **Geometric and Linear-Algebraic Intuition: From Stash Clutter to a 1D Board Game.** Even in rigorous mathematical analyses of *Diablo II* drop rates—such as `ShrimpDiablo`'s video *"Why you can't find Enigma"*—authors explicitly elide Horadric Cubing because enumerating every combination of partial rune drops makes direct combinatorial calculations appear intractable. Indeed, when a player asks *"How many runs until I can make a Lo rune?"*, naive enumeration explodes into a branching tree of mutually exclusive cases: finding 1 Lo directly, *or* 2 Ohms, *or* 1 Ohm+2 Vex, *or* 1 Ohm+1 Vex+2 Gul, *or* 1 Ohm+1 Vex+1 Gul+1 Ist+2 Mal, ad infinitum.[6]

However, the uniform 2-to-1 upgrade rule (14) collapses this entire branching tree of inventory combinations onto a **one-dimensional number line**—a simple board game that can be solved exactly using nothing more than elementary linear algebra (back-substitution on an upper-triangular matrix).

To see the geometry clearly before writing down the general case, consider a concrete miniature example: suppose a player is hunting a **Lo rune** and decides to pick up and cube only **Gul, Vex, Ohm, and Lo** runes. Because each tier requires exactly two of the previous tier (2 Gul = 1 Vex, 2 Vex = 1 Ohm, 2 Ohm = 1 Lo), we can measure every rune in **"Gul units"**:

$$\text{Gul} = 1 = 2^0, \qquad \text{Vex} = 2 = 2^1, \qquad \text{Ohm} = 4 = 2^2, \qquad \text{Lo} = 8 = 2^3.$$

Whenever two identical runes are cubed together (for example, 2 Vex → 1 Ohm, or 2+2 = 4), the operation is literally *carrying the* 1 *in base-*2 *(binary) addition.* Consequently, any combination of Gul, Vex, and Ohm runes in the player's stash is completely summarized by a single integer $V \in \{0, 1, 2, \dots, 8\}$ representing the player's total accumulated Gul units:

- If the player has 1 Ohm + 1 Vex + 1 Gul, their stash sits at square $V = 4 + 2 + 1 = 7$ on the board (in binary, $\texttt{0111}_2$).
- The moment a single additional Gul (+1) drops, the binary odometer rolls over ($\texttt{0111}_2 + \texttt{0001}_2 = \texttt{1000}_2 = 8$): 2 Gul → Vex, 2 Vex → Ohm, and 2 Ohm → Lo!

As illustrated in Figure 2(a), farming for a target rune is therefore equivalent to moving a token along a finite one-dimensional board $\{0, 1, \dots, W\}$ (where $W = 8$ for Gul → Lo, $W = 128$ for Pul → Lo, and $W = 512$ for Pul → Ber):

(1) **What makes it a Markov chain?** At the end of run n, your position on the board is some square $V_n \in \{0, 1, \dots, W\}$. Where you land on run $n+1$ depends *only* on your current square V_n and the random increment ΔV_{run} dropped in run $n+1$ ($V_{n+1} = \min(W, V_n + \Delta V_{\text{run}})$)—not on how many runs you spent reaching V_n.

(2) **What makes it an "absorbing" Markov chain?** Squares $0, 1, \dots, W-1$ are *transient*: the player has not yet crafted the target rune and continues running Lower Kurast. Square W is the *absorbing goal state*: once the player's stash reaches $\ge W$ units, the

[5]Because the gem-requiring Horadric Cube recipes begin at Thul (#10) and cycle through the six gem types (Topaz, Amethyst, Sapphire, Ruby, Emerald, Diamond) by quality tier, the first 2-to-1 upgrade—**Pul (#21) → Um (#22)**—is the final recipe of the **Flawed Gem** cycle (2 Pul + 1 Flawed Diamond → 1 Um), whereas Um (#22) through Ohm (#27) require Standard Gems (Topaz through Diamond) and Lo (#28) through Cham (#32) require Flawless Gems (Topaz through Emerald). Veteran Lower Kurast runners typically keep a small reserve of Flawed Diamonds on hand, since Hell superchests drop predominantly Standard and Flawless gems.

[6]While players intuitively recognize that cubing from El (#1) to Zod (#33) requires $3^{20} \cdot 2^{12} \approx 1.43 \times 10^{13}$ El runes—a timescale approaching the heat death of the universe—cubing from Pul (#21) to Lo (#28) or Ber (#30) spans only 7 to 9 binary steps ($2^7 = 128$ to $2^9 = 512$ Pul equivalents) and is entirely practical.

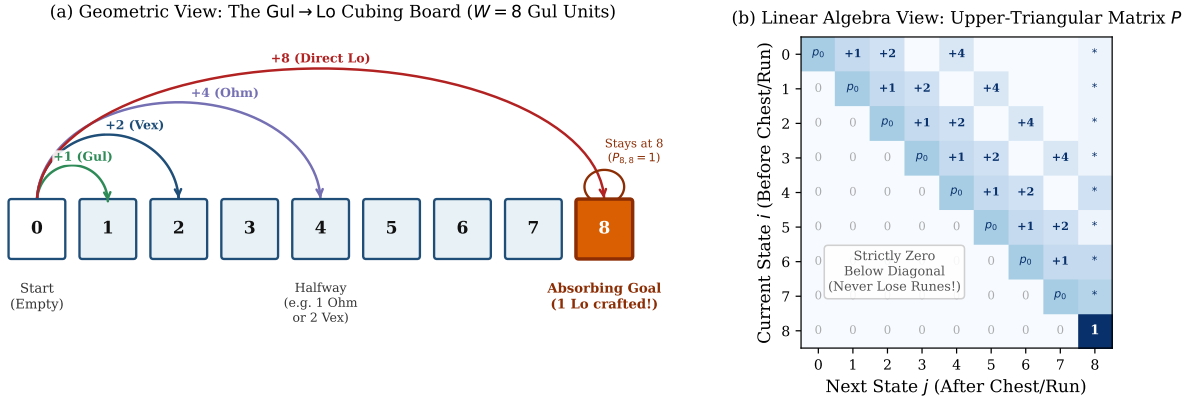


FIGURE 2. (a) Geometric representation of Horadric Cubing from Gul → Lo as a 1D board game on 9 squares ($V \in \{0, 1, \ldots, 8\}$ Gul units). Each chest or run advances the player's position forward by +1 (Gul), +2 (Vex), +4 (Ohm), or +8 (Lo), clamping at the absorbing goal square $W = 8$. (b) In linear algebra terms, because the player never loses runes, the state-transition matrix P is **upper triangular** ($P_{i,j} = 0$ for all $i > j$), allowing exact solution by back-substitution from Square 8 down to Square 0.

target rune is crafted and the farming project terminates (the token stays at Square W forever, $P_{W,W} = 1$).

(3) **How does linear algebra solve it?** Think of the probability distribution across the board after n runs as a row vector $\mathbf{u}^{(n)} = [u_0^{(n)}, u_1^{(n)}, \ldots, u_W^{(n)}]$, starting with 100% of the probability at Square 0: $\mathbf{u}^{(0)} = [1, 0, \ldots, 0]$. Each run simply multiplies this state vector by the $(W + 1) \times (W + 1)$ transition matrix P:

$$\mathbf{u}^{(n)} = \mathbf{u}^{(0)} P^n.$$

The last entry $u_W^{(n)}$—the fraction of probability mass that has been "absorbed" into the goal state W after n matrix multiplications—is the exact cumulative probability $F(n) = \mathbb{P}(R \leq n)$ that the player has crafted their rune within n runs! Better still, as shown in Figure 2(b), because a player never loses runes ($\Delta V_{\text{run}} \geq 0$), every entry below the main diagonal of P is zero ($P_{i,j} = 0$ for $i > j$). Thus, finding the expected number of runs to reach W from any square i is just solving an upper-triangular linear system by **back-substitution** starting from $E_W = 0$ and stepping backward to E_0.

6.2. **Formal Binary Radix Criterion and Backward Recurrences.** We now state and prove the general criterion for arbitrary starting and target runes. Fix a minimum rune tier $r_{\min} \in \{\text{Pul}, \ldots, r^*\}$ that the player is willing to pick up and cube toward a target rune $r^* \in \{\text{Vex}, \text{Ohm}, \text{Lo}, \text{Sur}, \text{Ber}\}$, and let $K = \text{idx}(r^*) - \text{idx}(r_{\min}) \geq 0$ denote the number of upgrade steps separating $r_{\min}$ from r^*.

Assign to each rune tier r_j ($\text{idx}(r_{\min}) \leq j \leq \text{idx}(r^*)$) its binary radix weight in units of $r_{\min}$:

$$v(r_j) = 2^{j - \text{idx}(r_{\min})} \in \{1, 2, 4, \ldots, 2^K\}, \qquad W := v(r^*) = 2^K. \tag{15}$$

Note that runes strictly higher than r^* (i.e., $j > \text{idx}(r^*)$) cannot be cubed downward in *Diablo II*; hence only drops in the range $\text{idx}(r_{\min}) \leq j \leq \text{idx}(r^*)$ contribute toward synthesizing r^*.

Proposition 6.1 (Binary Cubing Criterion)**.** *Let $c_0, c_1, \ldots, c_K \in \mathbb{Z}_{\geq 0}$ denote the inventory counts of runes from $r_{\min}$ ($k = 0$) to r^* ($k = K$). Repeated application of the* 2*-to-*1 *Horadric*

Cube recipe (14) *can synthesize at least one copy of* r^* *if and only if the total binary valuation satisfies*

$$V = \sum_{k=0}^{K} c_k \, 2^k \geq 2^K = W. \tag{16}$$

Proof. Necessity is immediate because each cubing operation replaces two runes of weight 2^k with one rune of weight 2^{k+1}, leaving the sum $V = \sum_{k=0}^{K} c_k 2^k$ invariant, and a single copy of r^* has weight 2^K. For sufficiency, proceed by induction on K. The base case $K = 0$ ($c_0 \geq 1$) is trivial. For $K \geq 1$, if $c_K \geq 1$ we are done; otherwise $c_K = 0$ and $\sum_{k=0}^{K-1} c_k 2^k \geq 2^K$. Performing $\lfloor c_0/2 \rfloor$ upgrades on tier 0 leaves at most one leftover rune at tier 0 ($c_0' = c_0 \bmod 2 \in \{0, 1\}$) and updates $c_1' = c_1 + \lfloor c_0/2 \rfloor$. Since $c_0' 2^0 \leq 1$ and the total sum is at least $2^K \geq 2$, the remaining sum over tiers $1 \leq k \leq K-1$ satisfies $\sum_{k=1}^{K-1} c_k' 2^k \geq 2^K$, or $\sum_{k=1}^{K-1} c_k' 2^{k-1} \geq 2^{K-1}$, which by the induction hypothesis cubes to r^*. □

By Proposition 6.1, the exact progress of a player cubing toward r^* from $r_{\min}$ is tracked by a discrete-time absorbing Markov chain [6] on the finite state space $\mathcal{S} = \{0, 1, \ldots, W\}$, where state $W = 2^K$ is absorbing. In a single chest opening, the valuation increment ΔV takes value 2^k with probability $p_{(k)} = w_{r_k}/N_{\text{pat}}$ ($0 \leq k \leq K$) and value 0 with probability $p_{(0)}^{\text{none}} = 1 - \sum_{k=0}^{K} p_{(k)}$. Over a 6-chest run, the run increment $\Delta V_{\text{run}} = \sum_{c=1}^{6} \Delta V_c$ has probability mass function $g(v) = \mathbb{P}(\Delta V_{\text{run}} = v)$ given by the 6-fold discrete convolution of the single-chest increment distribution.[7]

Because $\Delta V_{\text{run}} \geq 0$, the transition matrix on the transient states $\{0, 1, \ldots, W-1\}$ is upper triangular. Conditioning on the outcome of a single run from state $i < W$ yields $E_i = 1 + g(0)E_i + \sum_{v=1}^{W-1-i} g(v) E_{i+v}$ (since any jump $i + v \geq W$ lands in the absorbing state where $E_W = 0$). Solving for E_i (and similarly for the second moment $M_i = \mathbb{E}[R^2 \mid V_0 = i]$) gives the exact back-substitution formulas:

$$E_i = \frac{1 + \sum_{v=1}^{W-1-i} g(v)\, E_{i+v}}{1 - g(0)}, \tag{17}$$

$$M_i = \frac{2E_i - 1 + \sum_{v=1}^{W-1-i} g(v)\, M_{i+v}}{1 - g(0)}, \qquad \text{SD}(R) = \sqrt{M_0 - E_0^2}. \tag{18}$$

Table 1 (presented in Section 2) reports the exact expected run counts $E_0 = \mathbb{E}[R]$ and standard deviations $\text{SD}(R)$ computed from (17)–(18) for target runes $r^* \in \{\text{Mal}, \text{Ist}, \text{Gul}, \text{Vex}, \text{Ohm}, \text{Lo}, \text{Sur}, \text{Ber}\}$ under three cubing policies: (i) direct drops only ($r_{\min} = r^*$), (ii) cubing High Runes plus Gul ($r_{\min} = \text{Gul}$), and (iii) cubing all runes from Pul upward ($r_{\min} = \text{Pul}$).

6.3. **Why Naive Spreadsheet Division Underestimates Runs: The "Leftover Change" (Boundary Overshoot) Penalty.** When community guides and spreadsheets estimate the number of runs required to cube a target High Rune or complete *Enigma* (1 Jah+1 Ber = 3 Ber equivalents when cubed in Lower Kurast), they almost universally rely on a **naive linear-drift approximation**: adding up the average fractional value of every rune dropped per run into a single drift rate $\mathbb{E}[\Delta V_{\text{run}}]$ and dividing the target valuation W by that rate:

$$R_{\text{naive}} = \frac{W}{\mathbb{E}[\Delta V_{\text{run}}]}. \tag{19}$$

[7]We intentionally formulate the Markov chain at the granularity of complete 6-chest runs rather than individual chest clicks: in practice, no *Diablo II* player who drops their final rune on chest #2 immediately exits the game without opening the remaining four superchests in that 25-second run, and at the human level, players plan a Holy Grail project in terms of full runs and 1-to-2 hour weekend sessions rather than individual chest openings.

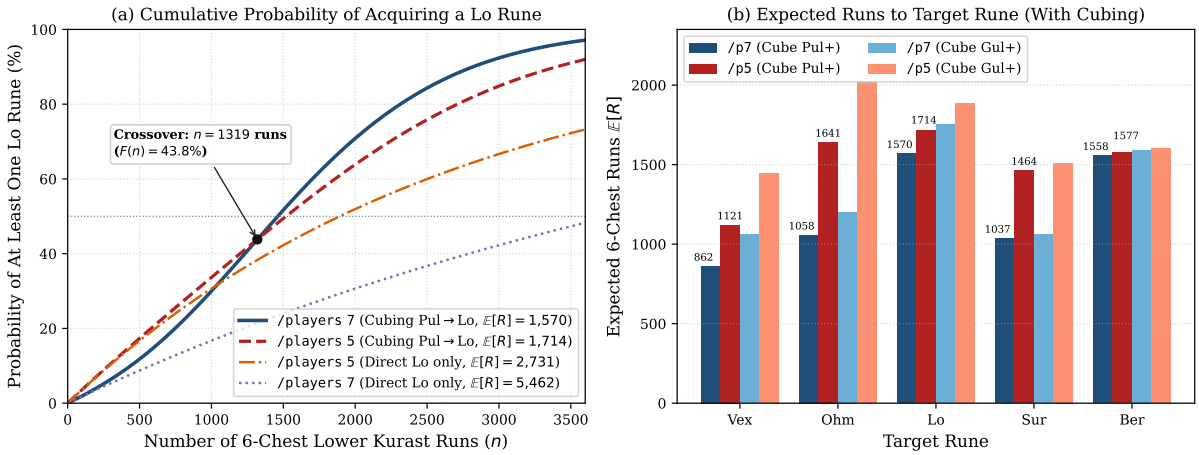


FIGURE 3. (a) Exact cumulative distribution function $F(n) = \mathbb{P}(R \leq n)$ of the number of 6-chest Lower Kurast runs required to acquire a Lo rune on `/players 7` vs. `/players 5`, comparing full Horadric Cubing (Pul → Lo) against direct Lo drops only. The cubing CDF on `/players 7` crosses above `/players 5` at $n = 1{,}319$ runs (43.8%) and exhibits substantially lower tail risk. (b) Expected 6-chest runs $\mathbb{E}[R]$ to acquire each High Rune (Vex–Ber) on `/players 7` vs. `/players 5` under Pul+ and Gul+ cubing.

On `/players 7` (cubing Pul+), this naive ratio yields 1,268.5 runs for a single Ber, 1,294.0 runs for a Lo, and the widely cited community benchmark of 3,805.6 runs for *Enigma*.

Yet as shown in the rightmost column of Table 1 (Section 2.1), our exact absorbing Markov chain reveals that the naive formula **systematically underestimates the true expected run count by** 21% **to** 26% **for single High Runes** (1,558.0 vs. 1,268.5 runs for Ber; 1,569.9 vs. 1,294.0 runs for Lo) and by +330.8 runs for *Enigma* (4,136.4 vs. 3,805.6 runs).

At first glance, the reason for this gap—known in stochastic processes as the **boundary overshoot penalty**—can feel counterintuitive. Suppose a player hunting a Lo rune sits at Square 7 on our Gul → Lo board (1 Ohm + 1 Vex + 1 Gul = $\frac{7}{8}$ Lo) and suddenly drops a natural Lo rune (+8 squares, landing at $V_R = 15$). A natural question arises: *"If a direct Lo drops while I am at Square* 7*, didn't I just finish my goal immediately? How could overshooting the boundary make the true expected run count* ***higher*** *than the naive estimate?"*

The answer is that dropping the natural Lo did not slow the player down; rather, **it rendered the $\frac{7}{8}$ Lo already sitting in the player's stash completely redundant for crafting that first Lo rune!** Look at what the naive formula (19) implicitly assumed when it added Ohm, Vex, Gul, and direct Lo drops together in the denominator $\mathbb{E}[\Delta V_{\text{run}}]$:

- By dividing 1.0 Lo ($W = 8$) by the combined drift rate of all runes from Pul to Lo, the naive spreadsheet assumes that 100% **of every rune ever picked up merges seamlessly into the target rune with zero "leftover change" in the stash**—as if a player holding $\frac{7}{8}$ Lo in cubed runes could combine them with $\frac{1}{8}$ of a natural Lo drop!
- In reality, runes arrive in discrete lumps. Whenever a player standing at Square 7 ($\frac{7}{8}$ Lo) finishes their goal by dropping a natural Lo (+1.0 Lo) or an Ohm (+0.5 Lo), they end the grind holding 1.875 Lo or 1.375 Lo in total value. The extra 0.875 Lo or 0.375 Lo sitting in their stash is **unused change** (overshoot $V_R - W > 0$) that contributed nothing to completing the target Lo rune, even though the naive formula gave those runes full credit in $\mathbb{E}[\Delta V_{\text{run}}]$.

A Concrete Two-Rune Miniature Example (Ohm + Lo). To see the mechanism with crystal clarity, consider a simplified Lower Kurast where only two runes exist:

- **Ohm** (= 0.5 Lo), dropping with probability 1/1,000 per run;
- **Lo** (= 1.0 Lo), dropping with probability 1/1,000 per run.

A player wants 1.0 Lo (either via 1 natural Lo or by cubing 2 Ohms).

(1) **What the naive spreadsheet predicts:** Each run drops an average of $\frac{1}{1{,}000}(0.5) + \frac{1}{1{,}000}(1.0) = \frac{1.5}{1{,}000}$ Lo per run. Dividing the 1.0 Lo target by this rate gives

$$R_{\text{naive}} = \frac{1.0 \text{ Lo}}{1.5/1{,}000 \text{ Lo/run}} = \mathbf{666.7} \text{ runs.}$$

(2) **What actually happens:** On any run, the probability of *some* rune dropping (either Ohm or Lo) is $\frac{2}{1{,}000} = \frac{1}{500}$, so the player waits an average of **500** runs just to see their *first* rune drop:
   - With probability 1/2, that first drop is a **Lo**: the player finishes immediately (500 runs).
   - With probability 1/2, that first drop is an **Ohm**: the player now holds 0.5 Lo and must wait an average of *another* **500** runs for a second rune drop (which will be either a second Ohm or a natural Lo, either of which completes the goal).

   Thus, the **true expected number of runs** is not 666.7, but

$$\mathbb{E}[R] = 500 + \frac{1}{2}(500) = \mathbf{750.0} \text{ runs} \quad (+12.5\% \text{ higher than } R_{\text{naive}}).$$

(3) **Where the extra** 83.3 **runs went:** Look at the 50% of players whose first drop was an Ohm (0.5 Lo). Half of them (25% of all players) finished on their second drop by finding a *second* Ohm (0.5 + 0.5 = 1.0 Lo, zero leftovers). The other half (25% of all players) were holding 1 Ohm when a *natural Lo* dropped! That 25% group finished the grind holding **1.5** Lo (1 Lo plus 1 unused Ohm in the stash). Across all players, the average total rune value sitting in the stash at the moment the grind ends is

$$\mathbb{E}[V_R] = 0.75\times(1.0 \text{ Lo}) + 0.25\times(1.5 \text{ Lo}) = \mathbf{1.125} \text{ Lo} \quad (\text{target 1.0 Lo} + \text{expected leftover 0.125 Lo}).$$

And how many runs does it take at the drift rate of 1.5 Lo per 1,000 runs to generate 1.125 Lo of total value?

$$\mathbb{E}[R] = \frac{\mathbb{E}[V_R]}{\mathbb{E}[\Delta V_{\text{run}}]} = \frac{1.0 \text{ Lo} + 0.125 \text{ Lo}}{1.5/1{,}000 \text{ Lo/run}} = \mathbf{750.0} \text{ runs.}$$

This exact identity—known in probability theory as **Wald's identity**, $\mathbb{E}[R] = \big(W + \mathbb{E}[V_R - W]\big)/\mathbb{E}[\Delta V_{\text{run}}]$—explains both why naive spreadsheets underestimate single-rune grinds by 21%–26% *and* why the percentage penalty shrinks for multi-Ber runewords in Table 1:

- When farming a **single Ber** ($W = 512$), the player stops the moment they cross 512, leaving an average of $\mathbb{E}[V_R - 512] = 0.228$ Ber ($\approx$ 117 Pul units) of unused "leftover change" in the stash (+22.8% overshoot penalty: 1,558.0 vs. 1,268.5 runs).
- When farming ***Infinity*** **(2 Ber)** or ***Enigma*** **(3 Ber)**, the leftover change sitting in the stash after completing Ber #1 is *not wasted*—it rolls directly over as a head start on Ber #2! Consequently, while the first Ber takes 1,558.0 expected runs, the second Ber requires only 2,865.3 − 1,558.0 = **1,307.3** additional runs, and the third Ber requires only 4,136.4 − 2,865.3 = **1,271.1** additional runs (nearly matching the naive drift rate of 1,268.5 runs), reducing the cumulative overshoot penalty from +22.8% down to +12.9% for *Infinity* and +8.7% for *Enigma*.

Remark 6.2 (Strategic Stash Corollaries: "Just-in-Time" Cubing and Preserving Scarce Lo Runes)**.** Combining Proposition 6.1 with the boundary-overshoot mechanism yields two practical rules for single-player stash management:

(1) **"Just-in-Time" (Lazy) Cubing:** Because the binary valuation $V = \sum_{k=0}^{K} c_k 2^k$ is invariant under (14), a player achieves the *exact same* Markov stopping time $R = \inf\{n : V_n \geq W\}$ by holding all Pul–Ohm runes **uncubed** in the stash until the exact run on which $V_n \geq W$. If a player hunting a Lo rune greedily cubes 2 Gul $\to$ Vex and 2 Vex $\to$ Ohm after every run and subsequently drops a natural Lo rune (boundary overshoot), they have irreversibly destroyed flexible Vex and Gul runes (needed for *Heart of the Oak*, *Mosaic*, or *Flickering Flame*) into a redundant second Ohm. Lazy cubing captures 100% of the Markov chain's speedup while keeping all "leftover change" in its most flexible uncubed denominations.

(2) **Withholding Scarce Lo Runes from Sur/Ber Cubing:** On an early single-player account, natural Lo drops are so rare on `/players 7` (2 patterns per chest vs. 9 for Sur) and so transformative for physical and mercenary builds (*Grief* and *Fortitude*) that veteran runners almost never cube 2 Lo $\to$ 1 Sur. Our upper-triangular recurrence (17)–(18) accommodates this policy directly: if a player withholds natural Lo drops ($p_{(\mathrm{Lo})} = 0$) while still cubing Pul–Ohm upward, expected runs on `/players 7` increase only modestly from 1,037.4 to 1,115.2 runs for Sur and from 1,558.0 to 1,659.1 runs for Ber (3,056.5 for *Infinity*; 4,404.4 for *Enigma*). Even if a player reserves *all* Pul–Lo drops to build *Heart of the Oak*, *Call to Arms*, *Grief*, and *Fortitude* in parallel and cubes only Sur + Ber ($r_{\min}$ = Sur) toward Ber-tier runewords, a single Ber takes 1,805.8 runs, *Infinity* takes 3,490.3 runs, and *Enigma* takes 5,170.7 runs.

7. Myth: "The Lo Rune Is More Likely on `/players 5`"

A pervasive piece of advice in Lower Kurast guides is that players seeking a **Lo rune** (essential for the *Grief* and *Fortitude* runewords) should switch from `/players 7` to `/players 5`. The superficial rationale is visible in the "Lo (#28)" row of Table 2: `/players 5` possesses 4 direct Lo patterns per chest, whereas `/players 7` possesses only 2. If a player refuses to use the Horadric Cube, direct Lo drops indeed occur twice as frequently on `/players 5` ($\mathbb{E}[R] = 2{,}731.0$ runs vs. 5,461.6 runs on `/players 7`).

However, no rational single-player farmer ignores Ohm, Vex, and Gul drops while hunting a Lo rune. When we apply the absorbing Markov chain framework of Section 6, the "`/players 5` for Lo" recommendation collapses completely:

Corollary 7.1 (Strict Dominance of `/players 7` for Lo Rune Acquisition)**.** *Under Horadric Cubing from either Gul upward ($r_{\min}$ = Gul) or Pul upward ($r_{\min}$ = Pul), `/players 7` strictly dominates `/players 5` for acquiring a Lo rune across all primary statistical metrics:*

(1) ***Lower Expected Runs:*** *Cubing from Pul upward requires* $\mathbb{E}[R_{p7}] = \mathbf{1{,}569.9}$ *runs on `/players 7` versus* $\mathbb{E}[R_{p5}] = \mathbf{1{,}714.2}$ *runs on `/players 5` (an* 8.4% *reduction in expected runs; cubing from Gul upward requires* 1,753.0 *vs.* 1,885.9 *runs).*

(2) ***Lower Median and Tail Risk (90th Percentile):*** *As shown in Figure 3(a), the cumulative distribution function* $F_{p7}(n) = \mathbb{P}(R_{p7} \leq n)$ *overtakes* $F_{p5}(n)$ *at* $n^* = 1{,}319$ *runs (*$F(1{,}319) = 43.8\%$*). Consequently, `/players 7` achieves both a lower median wait (***1,461** *vs.* **1,519** *runs) and a dramatically lower* 90*th-percentile drought (***2,820** *vs.* **3,401** *runs; standard deviation* 928.9 *vs.* 1,223.2 *runs).*

(3) ***Superior Sur and Overall High-Rune Yield on `/players 7`:*** *Because Sur and Ber cannot be cubed downward into Lo, their drops are orthogonal to the Lo Markov chain. However, on `/players 7`, the player simultaneously enjoys a* 66.7% *higher Sur drop rate*

*(*9.0 *vs.* 5.4 *patterns per chest) and a* 12.1% *higher total High Rune rate (*24.0 *vs.* 21.4 *Vex–Ber patterns per chest), plus extra leftover mid-runes (Mal, Ist, Gul) for* Call to Arms*,* Heart of the Oak*, and* Infinity*.*

Let us examine the mechanism behind Corollary 7.1. Why does `/players 7` beat `/players 5` for Lo despite having half as many direct Lo patterns? Consider the total **Lo-equivalent pattern weight** per chest from Pul through Lo, where each rune r_k is weighted by $2^{k-\mathrm{idx(Lo)}}$:

$$W_{\mathrm{Lo}}(\texttt{/p7}) = 2.0 + \frac{5.0}{2} + \frac{6.0}{4} + \frac{11.0}{8} + \frac{5.4}{16} + \frac{14.2}{32} + \frac{10.2}{64} + \frac{16.0}{128} = \mathbf{8.441}\ \text{Lo-patterns/chest}, \quad (20)$$

whereas on `/players 5`, the collapse of Ohm ($5 \to 2$) and Gul ($11 \to 5$) patterns reduces the Lo-equivalent drift to

$$W_{\mathrm{Lo}}(\texttt{/p5}) = 4.0 + \frac{2.0}{2} + \frac{6.0}{4} + \frac{5.0}{8} + \frac{5.2}{16} + \frac{10.0}{32} + \frac{11.0}{64} + \frac{16.4}{128} = \mathbf{8.063}\ \text{Lo-patterns/chest}. \quad (21)$$

On `/players 7`, the 5 Ohm patterns (= 2.5 Lo) and 11 Gul patterns (= 1.375 Lo) more than compensate for the 2 fewer direct Lo patterns, providing both a higher mean drift (8.441 vs. 8.063 Lo-equivalents per 65,534 chests) and a much smoother, lower-variance progression that avoids the 3,400-run dry spells of `/players 5`.

7.1. **Expanding the Scope: Universal Dominance Across Mal Through Ber.** Can this dominance of `/players 7` be expanded beyond the High Runes? When we evaluate the absorbing Markov chain for every mid-rune down to Pul (#21), a striking result emerges: **`/players 7` strictly dominates `/players 5` for every single rune from Mal (#23) through Ber (#30)**.

Theorem 7.2 (Universal Dominance of `/players 7` for Mal Through Ber)**.** *Under Horadric Cubing from Pul upward ($r_{\min}$ = Pul), `/players 7` strictly dominates `/players 5` for acquiring every target rune $r^* \in \{$Mal, Ist, Gul, Vex, Ohm, Lo, Sur, Ber$\}$ across all primary statistical measures:*

(1) ***Lower Expected Runs:*** *The expected number of* 6*-chest runs satisfies $\mathbb{E}[R_{p7}] < \mathbb{E}[R_{p5}]$ uniformly across the entire Mal–Ber range (Table 1). Specifically, `/players 7` reduces expected runs by* 15.6% *for Mal (*559.4 *vs.* 662.5 *runs),* 12.4% *for Ist (*768.2 *vs.* 876.5 *runs),* 35.1% *for Gul (*699.0 *vs.* 1,076.4 *runs),* 23.2% *for Vex (*861.7 *vs.* 1,121.4 *runs),* 35.5% *for Ohm (*1,057.8 *vs.* 1,641.1 *runs),* 8.4% *for Lo (*1,569.9 *vs.* 1,714.2 *runs),* 29.2% *for Sur (*1,037.4 *vs.* 1,464.4 *runs), and* 1.2% *for Ber (*1,558.0 *vs.* 1,576.9 *runs).*

(2) ***First-Order Stochastic Dominance for Mal, Ist, and Gul:*** *For target runes $r^* \in \{$Mal, Ist, Gul$\}$, the cumulative distribution functions satisfy*

$$F_{p7}(n) \geq F_{p5}(n) \quad \text{for all } n \geq 1, \quad (22)$$

*with strict inequality for all $n > 0$. Unlike the Lo rune—where `/players 5` enjoys a transient lead in cumulative probability before `/players 7` overtakes it at $n^* = 1{,}319$ runs—`/players 7` possesses strictly higher direct pattern counts for Mal (*14.2 *vs.* 10.0 *patterns), Ist (*5.4 *vs.* 5.2 *patterns), and Gul (*11.0 *vs.* 5.0 *patterns). As a result, `/players 7` leads at* ***every single run count*** *$n \geq 1$ without any crossover point, achieving both a lower median wait (Mal:* 446 *vs.* 548 *runs; Ist:* 670 *vs.* 771 *runs; Gul:* 586 *vs.* 966 *runs) and dramatically lower* 90*th-percentile tail risk (Mal:* 1,192 *vs.* 1,368 *runs; Ist:* 1,494 *vs.* 1,696 *runs; Gul:* 1,449 *vs.* 2,070 *runs).*

Remark 7.3 (The Pul and Um Exception and Why It Does Not Justify `/players 5`)**.** Only at the very base of the mid-rune ladder—**Pul (#21)** and **Um (#22)**—does `/players 5` exhibit slightly higher drop densities:

- **Pul (#21):** Unlocked and locked chest weighting yields $w_{\text{Pul}} = 16.4$ patterns on `/players 5` vs. 16.0 patterns on `/players 7`, translating to $\mathbb{E}[R] = 666.4$ runs on `/players 5` versus 683.1 runs on `/players 7` (a 2.5% advantage for `/players 5`). Because Pul is the base of the binary cubing lattice, cubing from Pul+ is identical to direct drops.
- **Um (#22):** `/players 5` possesses 11.0 patterns per chest versus 10.2 on `/players 7` (1 in 993.4 vs. 1 in 1,071.2 runs direct). When cubing Pul+, expected runs are $\mathbb{E}[R_{\text{p5}}] = 637.6$ runs (median 500) versus $\mathbb{E}[R_{\text{p7}}] = 671.9$ runs (median 528), giving `/players 5` a minor 5.1% edge (34.3 runs).

This shift occurs because the underlying `TreasureClassEx.txt` item-generation cascade allocates marginally more probability to lower item tiers on lower player counts.

However, this statistical footnote provides **zero justification** for running Lower Kurast on `/players 5`:

(1) **Hell Countess Is the Intended Mid-Rune Engine:** The Hell Countess possesses a special rune drop routine that drops up to 3 runes from `Countess Rune (H)` (cascading from `Runes 12` down to Ist) on `/players 1`, independent of superchest mechanics. Her exact direct drop rate is approximately 1 in 131 runs for Pul and 1 in 196 runs for Um (combined direct probability of ~ 1 in 78 runs for either Pul or Um), and when factoring in cubing from lower mid-runes (Lem/Fal/Ko) alongside direct drops, her average mid-rune yield is ≈ 0.070 Pul-equivalents per run (accumulating 1 Pul-equivalent of total rune wealth every ~ 14 runs, or 1 Um-equivalent every ~ 28 runs). By contrast, farming Lower Kurast for 670 runs (≈ 4.6 hours) specifically to acquire an Um rune is massively suboptimal regardless of player setting.

(2) **Catastrophic Opportunity Cost:** Dropping from `/players 7` to `/players 5` to shave 34 runs off an Um rune causes the player to forfeit a 42.0% boost in Mal runes, a 120.0% boost in Gul runes, a 150.0% boost in Ohm runes, a 66.7% boost in Sur runes, and a 12.1% boost in total High Runes. Lower Kurast is an engine for High Runes; running `/players 5` cripples that engine for a negligible gain on runes easily obtained elsewhere.

7.2. **Case Study: The 12,000-Run `/players 5` Illusion (`SirFoxGaming`).** The popular advocacy for farming Lower Kurast on `/players 5` reached its empirical zenith in content creator `SirFoxGaming`'s video, *12000 LOWER KURAST RUNS ON P5* [8], which cited an influential Reddit Monte Carlo analysis by `Slivenius` [9] purporting to show that `/players 5` optimizes high-rune targets like Ber and Lo.[8] The video thumbnail boldly proclaims: *"This is why you run Lower Kurast on Players 5."* Over an arduous endurance trial of $N = 12{,}000$ consecutive 6-chest runs (opening $m = 72{,}000$ superchests—representing approximately ~ 83.3 hours of frame-perfect 25-second farming), the creator acquired a large bounty of high runes, including direct Lo and Ber drops, presenting this haul to viewers as empirical validation of `/players 5`.

This argument commits a classic **attribution fallacy**: mistaking the sheer brute-force volume of a sample (72,000 trials) for the efficiency of the underlying distribution. In a universe of $N_{\text{pat}} = 65{,}534$ patterns, opening 72,000 superchests will shower any player in dozens of high runes simply by the Law of Large Numbers. The pertinent scientific question is not whether

[8] In his original simulation, `Slivenius` [9] initially omitted Gul runes entirely and incorporated non-superchest "poppables" (corpses, logs, skeletons) without accounting for the run-time dilation tax $\Delta\tau$ (Section 5.2). Even after adding an edit acknowledging that Gul runes close the gap to $\sim 1.5\%$, the model relied on continuous linear drift rather than discrete absorbing Markov cubing, overlooking how `/players 7`'s massive Ohm (+150%) and Gul (+120%) yields strictly overturn the direct Ber/Lo advantage across every rune from Mal through Ber.

12,000 runs yielded loot, but **how much expected rune value was permanently forfeited** by running those 12,000 runs on `/players 5` rather than `/players 7`.

Table 5 presents the exact expected drop balance sheet over 72,000 superchest openings across both player settings under the reverse-engineered locked/unlocked distribution.

TABLE 5. Expected rune drops over $N = 12{,}000$ runs (72,000 superchests opened) on `/players 5` vs. `/players 7`, demonstrating the net opportunity cost incurred by `SirFoxGaming` [8].

Rune	$\mathbb{E}$[Drops | `/p5`]	$\mathbb{E}$[Drops | `/p7`]	Net Difference (`p7` − `p5`)	Relative Change
Mal (#23)	10.99	15.60	**+4.61**	+42.0%
Ist (#24)	5.71	5.93	+0.22	+3.8%
Gul (#25)	5.49	12.09	**+6.59**	+120.0%
Vex (#26)	6.59	6.59	0.00	0.0%
Ohm (#27)	2.20	5.49	**+3.30**	+150.0%
Lo (#28)	4.39	2.20	−2.20	−50.0%
Sur (#29)	5.93	9.89	**+3.96**	+66.7%
Ber (#30)	4.39	2.20	−2.20	−50.0%
Total High Runes (Vex–Ber)	**23.51**	**26.37**	**+2.86**	**+12.1%**

The breakdown in Table 5 exposes the psychological cognitive trap behind `SirFoxGaming`'s conclusion:

(1) **The Direct Drop Saliency Bias:** Content creators are heavily biased by the visual thrill of seeing the exact string "Ber Rune" or "Lo Rune" drop on the ground. On `/players 5`, direct Ber and Lo runes drop twice as often (4.39 vs. 2.20 expected drops each). To an untrained observer, 4 direct Bers and 4 direct Los look like extraordinary wealth.

(2) **The Sur-to-Ber Cube Invariance:** What the creator failed to account for is that on `/players 7`, the chest table substitutes those 2.20 direct Bers with 3.96 **additional Sur runes** (9.89 vs. 5.93). In the Horadric Cube, 3.96 Sur = 1.98 Ber. Combining direct Ber drops with cubed Sur drops yields:

$$\text{Ber-equivalents}(\texttt{/p5}) = 4.39 + \frac{5.93}{2} = \mathbf{7.36}\ \text{Bers},$$
$$\text{Ber-equivalents}(\texttt{/p7}) = 2.20 + \frac{9.89}{2} = \mathbf{7.15}\ \text{Bers}.$$

Across an entire career of 12,000 runs, the difference in Ber-tier purchasing power between the two settings is a negligible 0.21 Ber ($\approx 1/5$th of a single Ber drop—a triviality entirely swallowed by binomial variance).

(3) **The Crippling Deficit in Endgame Runeword Currency:** While achieving parity on Ber-tier purchasing power, `SirFoxGaming`'s choice of `/players 5` imposed a staggering toll on his character's wider arsenal:

- **Ohm Runes (−3.30 forfeited, a 60.0% deficit):** `/players 7` produces 2.5× as many Ohm runes (5.49 vs. 2.20). That is more than **three missing *Call to Arms*** (or *Exile* / *Faith*) runewords.
- **Gul Runes (−6.59 forfeited, a 54.6% deficit):** `/players 7` produces more than double the Gul runes (12.09 vs. 5.49), representing more than **three additional *Heart of the Oak*** weapons via 2 Gul → 1 Vex.
- **Mal Runes (−4.61 forfeited):** Needed in pairs for *Call to Arms*, *Grief*, and *Infinity*.

- **Overall High Rune Deficit:** Summed across the entire Vex–Ber tier, running `/players 5` yielded nearly **three fewer High Runes** in total (23.51 vs. 26.37, a net deficit of 2.86 High Runes), as the surplus of Sur and Ohm on `/players 7` easily outweighed the direct Ber and Lo drops on `/players 5`.

Far from proving that "this is why you run Lower Kurast on `/players 5`," `SirFoxGaming`'s marathon stands as a cautionary tale: by spending 83 hours grinding on `/players 5`, he forfeited nearly 4 Sur runes, 3.3 Ohm runes, 6.6 Gul runes, and 4.6 Mal runes—all to chase the psychological dopamine of direct orange text that the Horadric Cube provides automatically on `/players 7`.

8. Conclusion

In unseeded single-player *Diablo II: Resurrected*, the Lower Kurast superchest grind is governed by a stationary Bernoulli process with $p_{\text{run}} \approx 1/455.51$ for Vex–Ber on `/players 7` under the exact locked/unlocked chest tables reverse-engineered by `Fruit` [4] and verified by `fearedbliss` [3]. Because the game's PRNG transitions are unweighted and initial session seeds are drawn from unknown system entropy, performing "random actions" prior to opening superchests merely applies a measure-preserving shift to a uniform distribution, leaving drop probabilities strictly invariant. Prolonged dry spells of 2,000 to 2,460 runs are 2.50σ–2.84σ Gaussian-equivalent tail events (1.23% to 0.45% single-wait probability, or 16.98% to 6.52% across a 15-High-Rune career), and the fact that `Dead_Sett`'s first High Rune dropped 461 runs after run 2,000 matches the unconditional memoryless expectation ($\mu_T = 455.51$ runs) to within 1.2%. Finally, when Horadric Cubing is modeled via an exact binary-radix absorbing Markov chain, `/players 7` strictly dominates `/players 5` in expected runs and tail risk for every rune from Mal through Ber—including the Lo rune (1,569.9 vs. 1,714.2 expected runs). While `/players 5` yields a trivial 5% edge for targeting isolated Pul or Um runes, targeting mid-runes in LK rather than at the Hell Countess is irrational, and doing so sacrifices the massive yield of Mal–Ber runes. Ultimately, the antidotes to a brutal Lower Kurast drought are not log-kicking rituals or `/players 5` detours, but a fast two-campfire map, a ruthless loot filter, and the Horadric Cube: the generator is not stuck, so keep teleporting, cube your Puls through Sur, and let the law of large numbers do the rest.

References

[1] Dead_Sett, *Watch this BEFORE you run Lower Kurast*, YouTube, 2024.

[2] W. Feller, *An Introduction to Probability Theory and Its Applications*, 3rd ed., vol. 1, John Wiley & Sons, New York, 1968.

[3] fearedbliss, *Lower Kurast Pattern Verification and Single-Player Drop Mechanics*, Diablo II Research and Archival Repository, 2020.

[4] Fruit (`bonqen`), *Diablo II Pattern Generator* (`d2pg`), Reverse engineering and exact drop pattern generation for superchests, 2018.

[5] Ginger Gaming Mentor, *D2R It's Raining Ber Runes: Seed Manipulation in Diablo II: Resurrected*, YouTube, 2023.

[6] J. G. Kemeny and J. L. Snell, *Finite Markov Chains*, D. Van Nostrand Company, Princeton, NJ, 1960.

[7] Nefarius, *The Phrozen Keep File Guides: `levels.txt` and `TreasureClassEx.txt`*, The Phrozen Keep Modding Knowledge Base, 2004–2006.

[8] SirFoxGaming, *12000 LOWER KURAST RUNS ON P5 (This is why you run Lower Kurast on Players 5)*, YouTube, 2023. https://www.youtube.com/watch?v=yPJIhJuISvg.

[9] Slivenius, *You are running LK wrong. (Statistical analysis)*, Reddit, `r/diablo2`, 2024. https://www.reddit.com/r/diablo2/comments/1g2eo10/you_are_running_lk_wrong_statistical_analysis/.

[10] TheHappyDev, *Lower Kurast (LK) Farming Guide and Drop Patterns*, `d2runes.io`, https://d2runes.io/rune-farming/lk-farming.